Weather Forecasting: Predicting Relative Humidity

Clarence Chow, Joel Ng, Vishnu, Dhathathreyan

1 Introduction

Accurately predicting relative humidity (RH) is vital for weather forecasting, public health, and environmental monitoring, but is challenging due to complex, nonlinear interactions among meteorological variables. To address this, we apply advanced machine learning methods—Random Forests, XGBoost, and Deep Learning. These methods are chosen for their ability to handle nonlinearities, multivariate dependencies, and feature interactions without strong parametric assumptions.

Recent work by Tao et al. (2022) compared a range of machine learning models for humidity prediction and found that ensemble-based methods such as Gradient Boosting and Random Forests outperformed traditional statistical models in accuracy. However, they noted drawbacks including computational cost and overfitting risks, particularly on small, noisy datasets. Deep neural networks, while powerful, were shown to require large datasets and intensive tuning. These were evident in the humidity studies by Hutapea et) and Frifra et al.(2024). These limitations inform our approach: leveraging both ensemble methods and deep learning to balance reliability and accuracy.

The objective of this study is to develop and evaluate machine learning models that effectively capture the relationships among environmental factors, that accurately predict RH at 1-hour, 6-hour, and 24-hour intervals.

2 Methodology

This study forecasts relative humidity (RH) using historical weather data collected near Monash University, Australia, from January 1, 2010, to May 31, 2021, comprising 100,057 hourly observations. The dataset includes features such as temperature, wind speed, pressure, solar and thermal radiation, and cloud cover.

The study adopts the following approach. We start with unsupervised learning, followed by data preprocessing, feature selection, and model development. Finally, we conclude by evaluating and comparing the different models. Attached below (Figure 1) is a flowchart of our Method Pipeline.

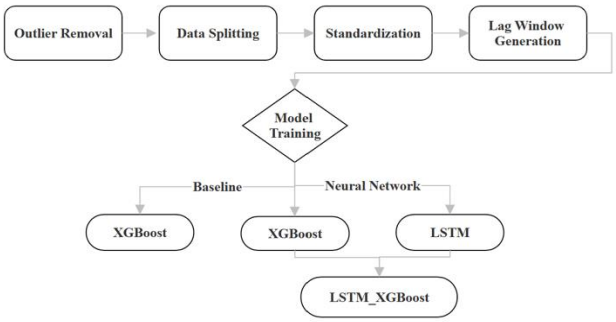


Figure 1: Flowchart of Method Pipeline

3 Unsupervised Learning

We first engage in data exploration to uncover underlying relationships in our dataset using unsupervised learning techniques such as Correlation Matrix, ACF, PACF, ADF, KPSS and Rolling Mean & SD techniques.

3.1 Correlation Map: A correlation matrix (Figure 2) was used to assess collinearity among the six input features. No strong correlations were found, with the highest being 0.66 between Temperature and Surface Thermal Radiation. This aligns with domain knowledge, and no features were dropped, as the lack of any high correlations suggests all inputs provide unique information for model training.

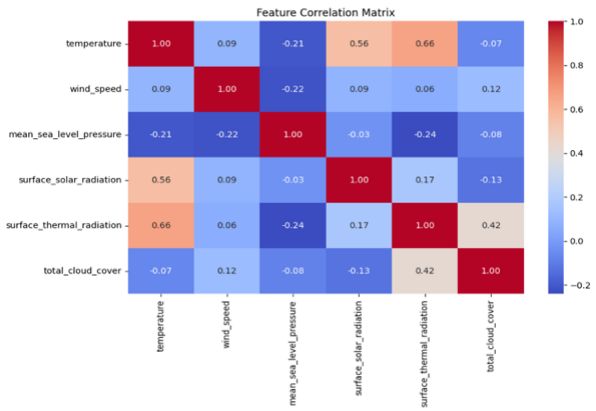


Figure 2: Correlation Heat Map

3.2 ACF and PACF: Using Autocorrelation Function (ACF) and Partial Autocorrelation Function (PACF), we explored patterns in our tine-series data, including seasonality and autocorrelation. The ACF plot for relative humidity displays a strong sinusoidal pattern (Figure 3), suggesting highly regular seasonal behavior, most likely on a 24-hour basis. The PACF plot shows a significant spike at lag 1 followed by a tapering pattern, indicating that the current RH level is

influenced by a strong autoregressive (AR) component at lags 1,2 and intervals of 24hrs.

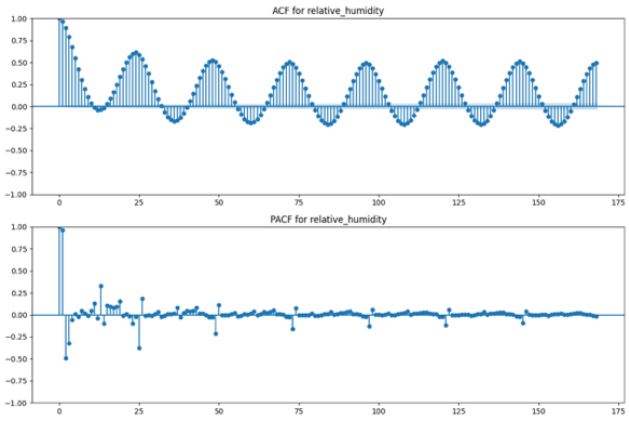


Figure 3: ACF and PACF (RH)

3.3 ADF and KPSS: We applied the Augmented Dickey-Fuller (ADF) and Kwiatkowski-Phillips-Schmidt-Shin (KPSS) tests to further assess stationarity in the time series data. Interestingly, the ADF test showed a statistic of -20.41 and a p-value of 0.0, indicating stationarity. However, the KPSS test statistic of 0.79 with a p-value of 0.01 suggesting non-stationarity. This contradiction suggests that the data is not trend stationary and possibly seasonal stationarity.

3.4 Rolling Mean and Standard Deviation: Rolling statistics help visually assess stationarity by evaluating if the mean and standard deviation remain constant over time. The rolling mean (red) fluctuates periodically, aligning with visible seasonal cycles in the data whilst the rolling standard deviation (green) also shows cyclical variation, further confirming seasonality and changing variance over time.

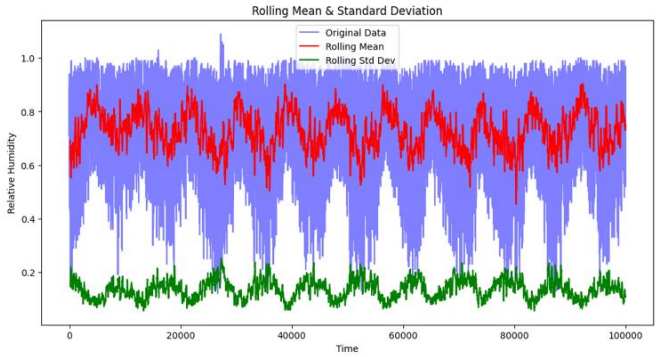


Figure 4: Rolling mean and SD

From our analysis of this relative humidity time-series, we conclude that our data is seasonality stationary, with strong seasonality peaking at 24 hours. As for model implications, our findings highlight the importance of using lag features for our models, and we have chosen to use N=24 as the optimal number of hours to train the model before making a future prediction in view of the strong daily seasonality.

4. Data Preprocessing

In the preprocessing stage, we perform key steps to prepare the dataset for effective model training and evaluation.

4.1 Removing Outliers: Relative humidity (RH) values exceeding 1 were removed, as they are physically impossible and likely due to sensor errors.

4.2 Standardizing: Features were standardized to zero mean and unit variance. Given the varied scales of weather variables, standardization helps stabilize learning—especially for models like LSTM that are sensitive to feature magnitude, aiding in pattern recognition performance.

4.3 Splitting Data: The 6 feature inputs(X) and the target feature (RH) were separated to prevent data leakage. This ensures predictions rely solely on prior weather inputs and not the actual target.

4.4 Creating Lag Windows: Lagged sequences were generated using a sliding window approach, capturing 1, 6, and 24-hour dependencies. These lag features help all three models—LSTM learns temporal dynamics, while XGB and Random Forest leverage past values to identify patterns without explicitly modeling time.

4.5 Train-test split: For XGBoost, we applied time series cross-validation using TimeSeriesSplit, respecting temporal order while ensuring robust evaluation. For LSTM, we used a standard 80-20 split. Validation for LSTM is discussed separately due to its bespoke training.

5 Models

5.1 Optimization

In our optimization step, we focus on improving the performance of our model using advanced techniques.

Bayesian Optimization with Optuna: We used Optuna for hyperparameter tuning, which applies Bayesian Optimization to efficiently explore the hyperparameter space. This probabilistic approach predicts promising areas based on prior evaluations, allowing for a more targeted search. Optuna focuses on key parameters like learning rate, batch size, and dropout rate, significantly reducing computational cost compared to grid or random search. Once the optimal combination is found, the model is retrained and evaluated to assess performance improvements.

5.2 Baseline Model (Random Forest)

To forecast relative humidity 1 hour ahead without using current or future humidity values, we framed the task as a regression problem using a Random Forest model. We engineered 1 to 24-hour lag features for temperature, wind speed,

pressure, radiation, and cloud cover to capture temporal patterns. After removing missing values, we applied MinMax scaling and split the data chronologically. Random Forest was chosen for its robustness, ability to handle non-linear relationships, and suitability for high-dimensional tabular data. It serves as a strong baseline by leveraging historical weather features for accurate humidity prediction.

5.3 Model 2 (XGBoost)

To improve upon the baseline, XGBoost was used for its ability to model complex, time-dependent patterns and its built-in regularization, which helps prevent overfitting. Timestamps were used to maintain chronological order, and seasonal features (month, weekday, season) were added to capture recurring humidity trends. Lag features (1, 6, 24 hours) and first-order differences for key variables (e.g., temperature, wind speed, cloud cover) were engineered to provide historical and temporal context.

Interaction terms and additional lagged variables like pressure and radiation were included to enrich the feature space. Rows with missing values were removed, and the target variable (RH) was shifted accordingly to avoid leakage. Data was split 80-20, with XGBoost trained using MAE and hyperparameters tuned via Optuna. Time series cross-validation ensured robust model evaluation.

5.4 Model 3 (LSTM)

Implementation: Our implementation of LSTM closely follows the model implemented by Hutapea et al. (2020) in a similar relative humidity study. To reduce computational demands, the model was built using Keras' Sequential API and follows a simple architecture consisting of LSTM–Dropout–LSTM–Dense layers. The model employs the RMSprop optimizer for weight updates, selected for its stability and improved control when tuning the learning rate.

To enhance predictive performance, Bayesian Optimization was applied to tune key hyperparameters (LSTM units, dropout rate, learning rate, batch size, epochs). This optimization process was carried out separately for the 1-hour, 6-hour, and 24-hour prediction models, resulting in a unique set of optimized hyperparameters for each forecasting horizon. Once the optimal configurations were determined, the models were trained using the dataset described in Section 4.5.

Cross-Validation: A 5-fold cross-validation strategy was employed during model training, specified within the [.fit()] method. The same validation loss metric was used as the objective function in Bayesian Optimization, guiding the selection of hyperparameters by minimizing validation error.

5.5 Hybrid Model (LSTM - XGBoost)

Implementation: The hybrid model builds upon the LSTM architecture trained in Section 5.4. Inspired by a storm prediction study conducted by Frifra et al. (2024), this approach leverages the LSTM's ability to learn complex non-linear relationships and temporal dependencies within the dataset. A new set of LSTM models is implemented by replicating the architecture and weights of the LSTM model from Section 5.4, excluding the final Dense layer. By omitting the output layer, we extract learned temporal features from the final LSTM layer without repeating the Bayesian optimization step.

When applied to the training data, this modified LSTM model generates a feature matrix, where each column corresponds to the output of an individual LSTM unit from the last LSTM layer. This matrix serves as a rich, time-aware representation of the input data, capturing temporal patterns essential for accurate forecasting.

The training feature matrix is then fed into a new XGB model. Bayesian Optimization is used to optimize XGB Hyperparameters. This optimization ensures that the XGB model is effectively calibrated to make accurate predictions using the LSTM-derived features, completing the training phase of the LSTM-XGB hybrid model.

Cross-Validation: Model training utilizes the cross_val_score function from sklearn.model_selection with a 5-fold cross-validation strategy. This approach ensures robustness and generalizability by evaluating performance across multiple data splits. The same cross-validation setup is used during Bayesian Optimization, where hyperparameters are tuned to minimize the cross-validation error, resulting in a well-regularized and high-performing hybrid model.

6 Evaluation and Discussion

Evaluation was conducted separately for each forecast horizon (1-hour, 6-hour, and 24-hour), using the respective test sets held out during training. We settled on using MAE as our main error metric as the values of Relative Humidity ranges from 0 to 1, hence MAE Gives the simplest representation of modelling error. The formula below describes how the metric is calculated.

$$MAE = \frac{\sum_{i=1}^{n} |y_i - \hat{y}|}{n}$$

$$y_i = actual\ values$$

$$\hat{y} = predicted\ values$$

$$n = number\ of\ observations$$

6.1 (Best) Models' Performance Evaluation

Model	1H (MAE)	6H (MAE)	24H (MAE)
RF	0.054	0.070	0.086
XGBoost	0.046	0.064	0.080
LSTM	0.047	0.065	0.084
LSTM + XGB	0.047	0.075	0.084

6.2 Comparative Analysis

After optimizing all four models through hyperparameter tuning and validation to minimize MAE, we found that Random Forest (RF) had the weakest performance (MAE = 0.054 at 1 hour), likely due to its lack of temporal awareness and inability to model sequential dependencies. In contrast, XGBoost delivered the best performance across all horizons (1H: 0.046, 6H: 0.064, 24H: 0.080). Its success can be attributed to its ability to model nonlinear relationships, handle high-dimensional lag features, and correct errors through sequential learning. Additionally, built-in regularization helps prevent overfitting on noisy weather data. LSTM models performed comparably at 1-hour forecasts but showed slightly higher errors for longer horizons, potentially due to limited training data or vanishing gradient issues. The LSTM + XGBoost hybrid did not outperform individual models, possibly due to integration complexity or redundant feature learning. Overall, XGBoost's tree-based structure excels at extracting patterns from engineered lag, difference, and interaction features, making it ideal for this tabular time-lagged dataset. LSTM performance could improve with more tuning or larger training windows.

6.3 Limitations

One key limitation is the exclusion of relative humidity as an input feature due to project constraints, which likely reduced model performance compared to real-world forecasting scenarios. Additionally, LSTM models were computationally intensive to train, and although Bayesian optimization improved performance, it required substantial resources. Finally, while seasonality was accounted for, extreme weather anomalies or sudden transitions may not be fully captured.

6.4 Suggestions for improvements

Future work can benefit from incorporating external weather sources (e.g., radar data or satellite imagery) to enhance long-term prediction accuracy. Additionally, experimenting with Transformer-based architectures may improve sequence modeling performance, especially for longer forecast windows. For real-world deployment, models can be retrained periodically with incoming data to adapt to climate shifts over time.

6.5 Performance against humans

Our models (MAE: 0.046-0.080) are comparable to human forecasters' 5-10% error rates. Models need not outperform humans to be useful—they provide complementary value through consistency and scalability. Privacy concerns are minimal as we use only environmental data, though geographic bias may exist when applied to different regions. While XGBoost offers some interpretability through feature importance, complex weather interactions remain challenging to explain. Rather than replacing meteorologists, these models will likely augment their work by handling routine forecasts, allowing humans to focus on complex events and communication.

7 References

Frifra, A., Maanan, M., Maanan, M., & Rhinane, H. (2024). Harnessing LSTM and XGBoost algorithms for storm prediction. *Scientific Reports*, *14*(1), 11381. https://doi.org/10.1038/s41598-024-62182-0

Hutapea, M. I., Pratiwi, Y. Y., Sarkis, I. M., Jaya, I. K., & Sinambela, M. (2020). *Prediction of relative humidity based on long short-term memory network*. 060003. https://doi.org/10.1063/5.0003171

Sharma, N. (2025). *How to use xgboost for Time-series forecasting?*. Analytics Vidhya. https://www.analyticsvidhya.com/blog/2024/01/xgboost-for-time-series-forecasting/

Tao, H., Awadh, S. M., Salih, S. Q., Shafik, S. S., & Yaseen, Z. M. (2022). Integration of extreme gradient boosting feature selection approach with machine learning models: Application of weather relative humidity prediction. *Neural Computing and Applications*, *34*(1), 515–533. https://doi.org/10.1007/s00521-021-06362-3